Phase 2 Complete Report: Web Application Security Risk Detection and Remediation Demonstration System Based on OWASP Top 10

Course Code: 03CIT4067 Cybersecurity Project
Project Title: Topic 15: Design and Implementation of a Web Application Security Risk Detection and Remediation Demonstration System Based on OWASP Top 10
Team: Group 7
Team Leader: WONG Lai Man (Max Wong) [Student ID: S24810493]
Team Members: MUHAMMAD MUSTAFA KHAN, Zoutao
Supervisor: Dr. Liam XU
Institution: Hong Kong College of Technology (HKCT)
Date: October 20, 2026

Executive Summary

This Phase 2 Report presents the comprehensive architecture, development, security testing, and management deliverables for Topic 15. Aligned strictly with the newly updated OWASP Top 10: 2025 security framework, the project addresses critical gaps in existing cybersecurity educational tools by providing an end-to-end platform covering both Red Team vulnerability detection and Blue Team code-level remediation.

The demonstration platform utilizes a hybrid system architecture combining a containerized instance of OWASP Juice Shop (deployed via Docker) for broad-spectrum vulnerability assessment with a custom-developed Python Flask demonstration engine. This custom engine specifically isolates complex business logic vulnerabilities, providing a side-by-side comparison of "Fail-Open" versus "Fail-Closed" behavior under the OWASP 2025 A10 category (Mishandling of Exceptional Conditions). Security testing methodologies integrate automated dynamic scanning via OWASP ZAP with targeted manual penetration testing using Burp Suite. Managed under an Agile Scrum framework, the project demonstrates effective risk mitigation, adaptive task reallocation, and full compliance with academic and professional standards.

Table of Contents

1. Chapter 1: Introduction
2. Chapter 2: Literature Review
3. Chapter 3: System Design & Methodology
4. Chapter 4: Agile Project Management
5. Chapter 5: Implementation & Demonstration
6. Chapter 6: Security Testing & Remediation
7. Chapter 7: Conclusion & Future Work
8. References
9. Appendices

Chapter 1: Introduction

1.1 Project Background

In the contemporary digital landscape, web applications serve as the primary interface for business operations, e-commerce, and data exchange. Consequently, they have become the most frequent target for cyberattacks. The Open Worldwide Application Security Project (OWASP) maintains the "OWASP Top 10," a standard awareness document representing the most critical security risks to web applications (OWASP Foundation, 2025a). As cloud-native architectures and complex supply chains become ubiquitous, the threat landscape has evolved significantly. The transition from the OWASP Top 10: 2021 to the newly released OWASP Top 10: 2025 reflects this shift, introducing new categories such as Software Supply Chain Failures (A03) and Mishandling of Exceptional Conditions (A10) (GitLab, 2026).

1.2 Problem Statement

Despite the widespread adoption of the OWASP Top 10 standard, existing educational and demonstration systems often rely on legacy vulnerabilities aligned with the 2021 edition. There is a distinct lack of comprehensive, hands-on demonstration platforms that specifically address the nuances of the 2025 updates. Furthermore, many existing tools focus solely on vulnerability detection (Red Team perspective) without providing clear, code-level remediation strategies (Blue Team perspective). This gap leaves developers and security practitioners without a practical framework to understand how to transition from identifying a flaw to implementing a secure, "fail-closed" architecture.

1.3 Project Objectives

The primary objective of this project is to design and implement a "Web Application Security Risk Detection and Remediation Demonstration System" (Topic 15). The specific goals are: 1. To deploy a modern, vulnerable web application environment (OWASP Juice Shop) that covers the majority of the OWASP Top 10: 2025 risks. 2. To develop custom, lightweight Python Flask modules specifically designed to demonstrate complex business logic flaws, particularly the A10: Mishandling of Exceptional Conditions (Fail-Open vs. Fail-Closed). 3. To integrate automated detection tools (OWASP ZAP) with manual penetration testing to validate vulnerabilities. 4. To provide a structured remediation framework, offering side-by-side comparisons of vulnerable and secure code for critical vulnerabilities.

1.4 Scope and Limitations

Scope: The project encompasses the deployment of OWASP Juice Shop via Docker, the development of custom Flask endpoints for A10 demonstrations, and the execution of Red/Blue team exercises covering Injection (A05), Broken Access Control (A01), and Exceptional Conditions (A10).
Limitations: The demonstration system operates within an isolated, local virtualized environment (Kali Linux/VirtualBox). While this ensures safety during penetration testing, it does not fully replicate the complexities of a production cloud environment, such as load balancers, Web Application Firewalls (WAF), or real-world network latency.

Chapter 2: Literature Review

2.1 Evolution of OWASP Top 10 (2021 to 2025)

The OWASP Top 10 is periodically updated to reflect the changing threat landscape. A comparative analysis of the 2021 and 2025 editions reveals critical shifts in industry priorities (OWASP Foundation, 2025a; GitLab, 2026): * **A01: Broken Access Control:** Remains the #1 risk in both editions. In 2025, Server-Side Request Forgery (SSRF) was folded into this category, emphasizing the need for strict server-side authorization and URL allowlists. * **A02: Security Misconfiguration:** Experienced the most dramatic jump, rising from #5 in 2021 to #2 in 2025. This is largely driven by the rapid adoption of cloud-native and containerized environments. * **A03: Software Supply Chain Failures:** An entirely new category at #3 in 2025, ranked #1 by the community survey. It expands beyond vulnerable components to cover the entire CI/CD pipeline. * **A10: Mishandling of Exceptional Conditions:** Another crucial new addition in 2025, covering "fail-open" vulnerabilities where system crashes inadvertently grant unauthorized access.

2.2 Web Application Security Risks

Web application risks generally stem from the improper handling of untrusted data. Injection flaws (A05), such as SQL Injection and Cross-Site Scripting (XSS), occur when untrusted data is sent to an interpreter as part of a command or query (OWASP Foundation, 2025a). Similarly, Broken Access Control (A01) allows attackers to act as users or administrators, often leading to unauthorized data disclosure or modification.

2.3 Remediation Strategies

Effective remediation requires a "Shift-Left" approach, integrating security early in the development lifecycle. * **For Injection (A05):** The industry standard is the use of Parameterized Queries (Prepared Statements) or Object-Relational Mapping (ORM) frameworks (OWASP Foundation, 2025a). For XSS, strict output encoding and Content Security Policy (CSP) headers are mandatory. * **For Exceptional Conditions (A10):** The core remediation strategy is implementing a "Fail-Closed" architecture. In the event of a database crash or unhandled exception, the system must default to denying access (e.g., returning HTTP 503) rather than bypassing security checks (Zenodo, 2025a). * **For Supply Chain (A03):** Remediation involves maintaining a Software Bill of Materials (SBOM), pinning dependency versions, and utilizing automated scanning tools like `npm audit` or `pip-audit` (Zenodo, 2025b).

Chapter 3: System Design & Methodology

3.1 System Architecture

To fulfill the "Detection and Remediation" requirement of Topic 15, the demonstration system is architecturally divided into two complementary components: 1. **Target Environment (Detection):** OWASP Juice Shop, deployed via Docker. This provides a comprehensive, modern Single Page Application (SPA) environment containing native vulnerabilities corresponding to 80% of the OWASP Top 10: 2025 categories. 2. **Custom Remediation Modules (Remediation):** A lightweight Python Flask application developed specifically to isolate and demonstrate complex logic flaws that are difficult to showcase in a pre-built target, most notably A10: Mishandling of Exceptional Conditions.

3.2 Technology Stack

The project utilizes a robust, industry-standard technology stack to ensure realistic simulation and safe execution: * **Virtualization & Containerization:** Oracle VirtualBox and Docker (for isolated deployment of `bkimminich/juice-shop`). * **Target Application:** OWASP Juice Shop (Node.js, Express, Angular, SQLite). * **Custom Module Development:** Python 3.10 and Flask (WSGI framework). * **Penetration Testing Environment:** Kali Linux (2026.x) running Mozilla Firefox, Burp Suite, and OWASP ZAP.

3.3 Threat Modeling (STRIDE Analysis)

Before implementation, a Threat Modeling exercise was conducted using the STRIDE methodology (Shostack, 2014; OWASP Foundation, 2023) to identify potential attack vectors and map them to our demonstration scope.

Table 3.1. STRIDE Threat Modeling Analysis.

STRIDE Category	Threat Description	Affected Component	OWASP Top 10: 2025 Mapping	Mitigation / Remediation Strategy
Tampering	**SQL Injection (SQLi):** Attacker injects payloads (e.g., `' OR 1=1--`) to manipulate database queries.	Login Endpoint (Juice Shop)	**A05: Injection**	Use Parameterized Queries or ORM (Sequelize). Never concatenate user input.
Tampering	**Cross-Site Scripting (XSS):** Malicious scripts injected into review fields and stored in the database.	Product Review Feature	**A05: Injection**	Implement strict Input Validation and Output Encoding (e.g., DOMPurify).
Elevation of Privilege	**Fail-Open Vulnerability:** System defaults to `return True` (Access Granted) when a backend exception occurs.	Custom Flask `/admin` endpoint	**A10: Mishandling of Exceptional Conditions**	Implement **Fail-Closed** mechanisms. The `except` block must default to denying access (HTTP 503).
Spoofing / Tampering	**Dependency Confusion:** Attacker publishes a malicious package to a public registry to hijack the build process.	CI/CD & Build Phase	**A03: Software Supply Chain Failures**	Use Software Bill of Materials (SBOM). Pin dependency versions and verify signatures.

STRIDE Category	Threat Description	Affected Component	OWASP Top 10: 2025 Mapping	Mitigation / Remediation Strategy
Information Disclosure	**Verbose Error Messages:** Application returns detailed stack traces, revealing internal framework versions.	Global Error Handler	**A02: Security Misconfiguration**	Disable debug mode in production. Implement global exception handlers returning generic messages.

Chapter 4: Agile Project Management

4.1 Agile Methodology Overview

To ensure the timely and high-quality delivery of the system, our team adopted the **Agile Scrum framework**. Unlike traditional waterfall models, Agile allows for iterative development, which is crucial for a cybersecurity project where threat landscapes evolve rapidly. We utilized **Jira** for Product Backlog management and **GitHub** for version control.

4.2 User Stories

User stories define the system requirements from the perspective of the end-users and stakeholders. The following three core user stories guided our Phase 2 development:

1. **Vulnerability Detection (Red Team Perspective)**
2. **As a** security tester,
3. **I want to** scan the target web application (OWASP Juice Shop) using automated tools and manual penetration testing techniques,
4. **So that** I can identify and document vulnerabilities mapped to the OWASP Top 10: 2025 standard.
5. **Remediation Demonstration (Blue Team/Developer Perspective)**
6. **As a** web developer,
7. **I want to** view a side-by-side comparison of vulnerable code and remediated code (e.g., Fail-Open vs. Fail-Closed mechanisms),
8. **So that** I can understand the root cause of the security flaws and implement effective patches.
9. **Risk Visualization (Stakeholder Perspective)**
10. **As a** project stakeholder,
11. **I want to** access a centralized dashboard displaying the detection results and remediation status,
12. **So that** I can evaluate the overall security posture of the demonstration system without needing deep technical expertise.

4.3 Work Breakdown Structure (WBS)

The WBS decomposes the project scope into manageable deliverables.

Table 4.1. Work Breakdown Structure (WBS).

WBS ID	Task Name	Assignee	Status
1.0	**Phase 1: Proposal & Inception**	**Team**	**Completed**
1.1	Draft Project Proposal & Scope Definition	WONG Lai Man	Completed
1.2	Literature Review (OWASP 2021 vs. 2025)	Mustafa	Completed
2.0	**Phase 2: System Design & Implementation**	**Team**	**In Progress**
2.1	Environment Setup (Docker & Juice Shop)	Mustafa	Completed
2.2	Red Team Exercise & Attack Documentation	Mustafa	Completed
2.3	Threat Modeling (STRIDE Analysis)	WONG Lai Man	Completed
2.4	Custom Flask Module Development (A10)	Mustafa / WONG Lai Man	Completed
2.5	Red Team Operation Logs (A01-A10)	Zoutao	Completed

4.4 Two-Term Gantt Chart Roadmap

- **Term 1 (Weeks 1-10):** Project Inception, Topic Selection, Environment Setup, Literature Review, and Initial Red/Blue Team Exercise.
- **Term 2 (Weeks 11-20):** Full System Integration, Automated Scanning (OWASP ZAP), Remediation Dashboard Development, Final Report Compilation, and Presentation.

4.5 Sprint 1 Execution & Risk Management

During Sprint 1, the team successfully deployed the OWASP Juice Shop environment. Midway through Phase 2, the project faced resource constraints due to a team member's health and network issues. The Team Leader initiated a **task reallocation strategy**, absorbing the Threat Modeling workload and integrating the member's written Red Team logs to ensure the project remained on schedule.

Chapter 5: Implementation & Demonstration

5.1 Development Environment & Tools

- **Target Vulnerable Application:** OWASP Juice Shop (deployed via Docker container `bkimminich/juice-shop`).
- **Custom Remediation Modules:** Python 3.10 and Flask.
- **Operating System & Testing Environment:** Kali Linux (VirtualBox).
- **Web Browser:** Mozilla Firefox.

5.2 System Architecture Implementation

The demonstration system is architecturally divided into two core components: 1. **Detection Environment (Red Team):** The OWASP Juice Shop instance serves as the primary target for vulnerability detection. 2. **Remediation Dashboard (Blue Team):** A custom Python Flask application was developed to act as a controlled environment for A10 demonstrations.

5.3 Vulnerability Detection & Remediation Demonstration

5.3.1 A05: Injection - SQL Injection (Authentication Bypass)

Detection (Attack Execution): By inputting the classic SQLi payload `' OR 1=1--` into the email field, the attacker manipulates the backend SQL query.

[Insert Figure 5.1 here: Fig5.1_A05_SQLi_PayloadInput.png]
Figure 5.1. SQL injection payload executed on the OWASP Juice Shop login page.

As shown in Figure 5.2, the application fails to sanitize the input, successfully bypassing the authentication logic.

[Insert Figure 5.2 here: Fig5.2_A05_SQLi_AuthBypassSuccess.png]
Figure 5.2. Successful authentication bypass demonstrating the SQL injection vulnerability.

Remediation Strategy: * Vulnerable Code: `db.query("SELECT * FROM Users WHERE email = '" + input + "'");` * Remediated Code (Node.js/Sequelize): `User.findOne({ where: { email: input } });`

5.3.2 A05: Injection - Stored Cross-Site Scripting (XSS)

Detection (Attack Execution): We targeted the "Product Review" feature. An attacker can inject malicious HTML tags, such as `<iframe src="javascript:alert('XSS')"></iframe>`, into the review comment field.

[Insert Figure 5.3 here: Fig5.3_A05_XSS_StoredPayloadInput.png] Figure 5.3. Malicious HTML payload submitted into the product review section.

[Insert Figure 5.4 here: Fig5.4_A05_XSS_UnsanitizedRender.png] Figure 5.4. Unsanitized HTML tags rendered in the browser, confirming the stored XSS vulnerability.

Remediation Strategy: * Vulnerable Code: `element.innerHTML = userInput;` * Remediated Code: `element.textContent = userInput;` or `DOMPurify.sanitize(userInput);`

5.3.3 A10: Mishandling of Exceptional Conditions (Fail-Open vs. Fail-Closed)

Note: This is a critical new category emphasized in the OWASP Top 10: 2025 updates.

Detection (Fail-Open Demonstration): We developed a custom Flask module (`a10_demo.py`). The `/admin` route checks the user role. If an exception occurs, the `except` block incorrectly returns "Access Granted".

[Insert Figure 5.5 here: Fig5.5_A10_VulnerableCode_Nano.png] Figure 5.5. Vulnerable Python Flask code demonstrating the fail-open mechanism.

[Insert Figure 5.6 here: Fig5.6_A10_FailOpen_BrowserResult.png] Figure 5.6. Browser output showing unauthorized access granted due to an unhandled exception.

Remediation Strategy (Fail-Closed Implementation): The secure approach is "Fail-Closed". In the event of an error, the system must default to denying access (HTTP 503).

[Insert Figure 5.7 here: Fig5.7_A10_RemediatedCode_Nano.png] Figure 5.7. Remediated Flask code implementing a fail-closed mechanism.

[Insert Figure 5.8 here: Fig5.8_A10_FailClosed_BrowserResult.png] Figure 5.8. Browser output showing secure default behavior (access denied) after remediation.

5.4 Automated Security Scanning Integration

To fulfill the "Detection System" requirement, manual penetration testing was supplemented with automated scanning using **OWASP ZAP**. The generated HTML reports serve as the foundational data for our risk detection dashboard.

Chapter 6: Security Testing & Remediation

6.1 Security Testing Methodology

Our testing strategy integrates **Automated Vulnerability Scanning** (OWASP ZAP) and **Manual Penetration Testing** (Burp Suite), aligned with the OWASP Testing Guide v4 (OWASP Foundation, 2025b).

6.2 Automated Vulnerability Detection (OWASP ZAP Integration)

6.2.1 Scanning Configuration

1. **Proxy Setup:** OWASP ZAP configured as a local proxy on port 8080.
2. **Spidering:** Mapping the application's attack surface.
3. **Active Scan:** Injecting payloads to observe responses.

6.2.2 Risk Analysis & Reporting

- **High Risk:** SQL Injection (A05) and Broken Access Control (A01).
- **Limitation:** Automated scanners failed to identify the A10 logic flaw, necessitating manual testing.

6.3 Manual Penetration Testing & Validation

Using Burp Suite's Repeater tool, we manually crafted HTTP GET requests to the `/admin` endpoint (`?user=attacker`), forcing a simulated database exception to validate the Fail-Open and Fail-Closed states.

6.4 Comprehensive Remediation Framework

1. **Input Validation and Output Encoding (A05):** Enforce Parameterized Queries and DOMPurify.
2. **Fail-Closed Architecture Design (A10):** Redesign error handling to deny access by default.
3. **Software Supply Chain Security (A03):** Integrate `npm audit` and `pip-audit`.

Chapter 7: Conclusion & Future Work

7.1 Project Summary

This project successfully designed and implemented a demonstration system aligned with the OWASP Top 10: 2025 standard, bridging the gap between theoretical vulnerability analysis and practical code-level remediation.

7.2 Key Achievements

1. **Alignment with OWASP 2025 Standards:** Explicitly addressing A10 (Fail-Open vs. Fail-Closed).
2. **Hybrid Detection Methodology:** Combining OWASP ZAP with manual Burp Suite testing.
3. **Actionable Remediation Framework:** Providing side-by-side code comparisons.
4. **Effective Agile Risk Management:** Successfully navigating resource constraints.

7.3 Limitations

1. **Scope of Automated Detection:** DAST tools struggle with complex business logic (A10).
2. **Environment Isolation:** Local Docker environment lacks production complexities (WAF, Load Balancers).

7.4 Future Work

1. **Integration of SAST and SCA Tools:** Shift-Left security view.
2. **Cloud-Native Misconfiguration Module:** Demonstrating A02 on AWS/Azure.
3. **Automated CI/CD Security Pipeline:** Enforcing security checks via GitHub Actions.

References

GitLab. (2026, January 7). *OWASP Top 10 2025: What's changed and why it matters*. GitLab Blog. https://about.gitlab.com/blog/2025-owasp-top-10-whats-changed-and-why-it-matters/

IBM Security. (2025). *Cost of a data breach report 2025*. IBM Corporation. https://www.ibm.com/reports/data-breach

OWASP Foundation. (2021). *OWASP Top 10:2021: The ten most critical web application security risks*. https://owasp.org/www-project-top-ten/

OWASP Foundation. (2023). *OWASP Threat Modeling Cheat Sheet*. https://cheatsheetseries.owasp.org/cheatsheets/Threat_Modeling_Cheat_Sheet.html

OWASP Foundation. (2025a). *OWASP Top 10:2025: The ten most critical web application security risks*. https://owasp.org/www-project-top-ten/

OWASP Foundation. (2025b). *OWASP Testing Guide v4*. https://owasp.org/www-project-web-security-testing-guide/

Schadler, B., & Klein, A. (2024). *OWASP Juice Shop: Vulnerable web application for security training*. https://owasp.org/www-project-juice-shop/

Shostack, A. (2014). *Threat modeling: Designing for security*. John Wiley & Sons.

Zenodo. (2025a). *Mishandling of exceptional conditions vulnerability: A detailed study*. https://zenodo.org/records/17867093

Zenodo. (2025b). *Software supply chain failures: A detailed vulnerability study with applicable remediations for PHP-based applications*. https://zenodo.org/records/17866839

Appendices

Appendix A: Red/Blue Team Exercise Report

Document Reference: Red/Blue Team Security Assessment Report **Primary Author:** MUHAMMAD MUSTAFA KHAN **Target Application:** OWASP Juice Shop & Custom Demonstration Module

A.1 Overview

This appendix incorporates the baseline security findings compiled during initial Sprint 1 Red Team exercises, providing technical verification for SQL Injection (A05), Stored XSS (A05), and Exceptional Conditions (A10).

A.2 Technical Execution Summaries

1. **SQL Injection (Auth Bypass):** Executed via `' OR 1=1--`. Confirmed administrative session token issuance.
2. **Stored Cross-Site Scripting:** Injected HTML payloads into product review storage. Verified execution upon DOM rendering.
3. **Exceptional Conditions (Fail-Open):** Simulated database failure in Flask backend. Verified unhandled exception returned HTTP 200 with authorization flags enabled.

Appendix B: Red Team Operation Logs (A01–A10)

Document Title: Red Team Vulnerability Assessment & Operation Logs (OWASP Top 10: 2025 Framework) **Target Environment:** OWASP Juice Shop (v14.5.0) & Custom Flask Demonstration Engine **Contributor:** Zoutao (Operation Steps & Logs) **Editor & Academic Standardizer:** WONG Lai Man (Team Leader)

B.1 Executive Overview

To validate the security posture against the OWASP Top 10: 2025 standard, Red Team operations were performed across ten vulnerability vectors. This appendix documents technical operational steps, threat impact analysis, and Blue Team remediation principles.

B.2 Detailed Category Logs

Category A01: Broken Access Control

- **Vulnerability Description:** Failure to enforce vertical access controls on administrative endpoints.
- **Operational Execution Steps:**
- Register a standard user account and log in.
- Navigate directly to `http://localhost:3000/#/administration`.
- Observe loading of the Administrative Dashboard without server-side permission verification.
- **Remediation Strategy:** Enforce strict server-side Role-Based Access Control (RBAC) and implement client-side route guards.

Category A02: Security Misconfiguration

- **Vulnerability Description:** Unsafe default server configurations and permissive file upload validation rules.
- **Operational Execution Steps:**
- Navigate to the Customer Complaint section.
- Create and upload an XML payload file (`test.xml`).
- Observe successful server processing without file type restriction.
- **Remediation Strategy:** Implement strict file extension allowlisting, MIME-type verification, and disable XML external entity resolution.

Category A03: Software Supply Chain Failures

- **Vulnerability Description:** Reliance on third-party libraries containing known, unpatched CVEs.
- **Operational Execution Steps:**
- Inspect browser Developer Tools (Sources tab) for `lodash`.
- Identify version `4.17.21` and map to known Prototype Pollution vulnerability.
- **Remediation Strategy:** Integrate automated dependency scanning (`npm audit`, `Snyk`) and maintain a Software Bill of Materials (SBOM).

Category A04: Cryptographic Failures

- **Vulnerability Description:** Insecure storage of sensitive credentials using legacy hashing algorithms (MD5).
- **Operational Execution Steps:**
- Inspect exported database tables containing user credentials.
- Identify unsalted MD5 hashes and pass through rainbow table service to recover plaintext.
- **Remediation Strategy:** Replace legacy hashes with Argon2id or bcrypt. Enforce

TLS/HTTPS.

Category A05: Injection (SQL Injection)

- **Vulnerability Description:** Dynamic SQL query string concatenation without input sanitization.
- **Operational Execution Steps:**
- Navigate to product search field.
- Inject payload: `' OR 1=1--`.
- Observe query bypass returning all database records.
- **Remediation Strategy:** Enforce Prepared Statements or ORM frameworks. Apply principle of least privilege.

Category A06: Insecure Design

- **Vulnerability Description:** Flaws in business logic where critical transaction parameters are trusted from the client side.
- **Operational Execution Steps:**
- Intercept HTTP POST request during checkout using Burp Suite.
- Modify `price` parameter to `0`.
- Forward request and observe successful order placement at zero cost.
- **Remediation Strategy:** Perform all business logic validation and price calculations strictly on the server side.

Category A07: Authentication Failures

- **Vulnerability Description:** Missing rate limiting and weak password complexity policies.
- **Operational Execution Steps:**
- Target administrative login interface (`admin@juice-sh.op`).
- Execute automated credential brute-force attack.
- Observe authentication without rate limits or MFA challenges.
- **Remediation Strategy:** Enforce MFA and implement progressive delay and rate limiting on failed attempts.

Category A08: Software and Data Integrity Failures

- **Vulnerability Description:** Accepting code or payloads without enforcing cryptographic signature verification.
- **Operational Execution Steps:**
- Access file update endpoint.
- Modify file content in transit and submit.
- Observe server processing without checking digital signatures.
- **Remediation Strategy:** Implement digital signatures (GPG/code signing) and verify SHA-256 hashes.

Category A09: Security Logging and Monitoring Failures

- **Vulnerability Description:** Security-relevant events are neither logged nor monitored.
- **Operational Execution Steps:**

- Conduct high-volume brute-force and SQL injection tests.
- Inspect system monitoring; observe no log entries or alert notifications.
- **Remediation Strategy:** Implement centralized logging and establish automated SIEM alerting rules.

Category A10: Server-Side Request Forgery (SSRF) / Exceptional Conditions

- **Vulnerability Description:** Fetching remote resources based on user-supplied URLs without restricting requests to private IP ranges.
- **Operational Execution Steps:**
- Locate remote profile image URL submission feature.
- Input internal loopback address: `http://127.0.0.1:3000`.
- Observe application making internal HTTP request and returning local service data.
- **Remediation Strategy:** Enforce strict URL domain allowlisting and block requests targeting loopback/private RFC 1918 IP spaces.

Appendix C: Communication Logs & Task Coordination Record

Course & Project: 03CIT4067 Cybersecurity Project - Group 7 (Topic 15) **Record Keeper:** WONG Lai Man (Team Leader)

C.1 Initial Communication & Context Clarification (October 16-20, 2026)

- **Issue:** Initial communication delay regarding meeting availability.
- **Resolution:** Contacted via WhatsApp on October 20. Zoutao clarified hospitalization and network connectivity issues in Mainland China.

C.2 Technical Constraints & Task Reallocation (October 20, 2026)

- **Challenge:** Inability to deploy OWASP Juice Shop via Docker due to regional network restrictions.
- **Mitigation Strategy:** Reallocated local deployment to Mustafa. Zoutao reassigned to documentation and Red Team operation logs.

C.3 Written Deliverables Received from Zoutao

- Zoutao successfully submitted two text files containing detailed Red Team operation steps for OWASP Top 10 categories (A01-A10).
- These documents are integrated into Appendix B of the Phase 2 Report.

Declaration

I, WONG Lai Man, declare that the above records reflect the team's agile adaptation to member constraints. Zoutao has contributed written deliverables despite technical barriers.

Signature: _____ (WONG Lai Man - Team Leader) **Date:** October 20, 2026